문제 1) 이진 트리의 순회 및 복사 구현

문제 설명

이진 트리(Binary Tree)에 대하여 다음 기능을 완성하시오.

- theInorder(BTNode t) : 재귀를 이용하여 중위 순회(Inorder Traversal)를 수행한다.
- thePreorder(BTNode t) : 재귀를 이용하여 전위 순회(Preorder Traversal)를 수행한다.
- thePostorder(BTNode t) : 재귀를 이용하여 후위 순회(Postorder Traversal)를 수행한다.
- inorderIter1() : Stack을 이용하여 반복적인 중위 순회를 수행한다.
- preorderIter() : Stack을 이용하여 반복적인 전위 순회를 수행한다.
- levelorder() : Queue를 이용하여 레벨 순회(Level-order Traversal)를 수행한다.
- copy() : 현재 이진 트리를 복사한 새로운 BinaryTree 객체를 반환한다.
- theCopy(BTNode t) : 재귀를 이용하여 각 노드를 복사한다.
- theEqual(BTNode s, BTNode t) : 두 이진 트리의 구조와 각 노드의 데이터가 동일한지 비교한다.

Stack, Queue, BTNode, BinaryTree의 기본 연산 및 main( ) 함수는 주어진 코드를 사용한다.

기본 코드

```
class BTNode {
    String data;
    BTNode Lchild;
    BTNode Rchild;
    public BTNode() {
    }
    public BTNode(String dt) {
        data = dt;
        Lchild = null;
```

```
        Rchild = null;
    }
    public BTNode(BTNode lc, String dt, BTNode rc) {
        data = dt;
        Lchild = lc;
        Rchild = rc;
    }
}
class BinaryTree {
    BTNode root;
    public BinaryTree() {
        root = null;
    }
    public BinaryTree(BinaryTree ltree, String data, BinaryTree rtree) {
        root = new BTNode(ltree.root, data, rtree.root);
    }
    public boolean isEmpty() {
        return root == null;
    }
    public BinaryTree leftSubTree() {
        BinaryTree ltree = new BinaryTree();
        ltree.root = root.Lchild;
        return ltree;
    }
    public BinaryTree rightSubTree() {
        BinaryTree rtree = new BinaryTree();
```

```
        rtree.root = root.Rchild;
        return rtree;
    }
    public String rootData() {
        return root.data;
    }
    public void inorder() {
        System.out.println();
        System.out.println("Inorder");
        theInorder(root);
        System.out.println();
        System.out.println("InorderIter");
        inorderIter1();
    }
    public void preorder() {
        System.out.println("preorder");
        System.out.println();
        thePreorder(root);
        System.out.println();
        System.out.println("preorderIter");
        preorderIter();
    }
    public void postorder() {
        thePostorder(root);
    }
    private void theInorder(BTNode t) {
```

```
        // 코드 작성
    }
    private void thePreorder(BTNode t) {
        // 코드 작성
    }
    private void thePostorder(BTNode t) {
        // 코드 작성
    }
    private void inorderIter1() {
        // 코드 작성
    }
    private void preorderIter() {
        // 코드 작성
    }
    public void levelorder() {
        // 코드 작성
    }
    public BinaryTree copy() {
        // 코드 작성
    }
    private BTNode theCopy(BTNode t) {
        // 코드 작성
    }
    public boolean equals(BinaryTree tr) {
        return theEqual(this.root, tr.root);
    }
```

```
    private boolean theEqual(BTNode s, BTNode t) {
        // 코드 작성
    }
}
public class BinaryTreeTest {
    public static void main(String args[]) {
        BinaryTree btree;
        BinaryTree ltree;
        BinaryTree rtree;
        BinaryTree current;
        rtree = new BinaryTree(new BinaryTree(), "H", new BinaryTree());
        btree = new BinaryTree(new BinaryTree(), "G", rtree);
        ltree = btree;
        btree = new BinaryTree(ltree, "E", new BinaryTree());
        ltree = new BinaryTree(new BinaryTree(), "D", new BinaryTree());
        rtree = btree;
        btree = new BinaryTree(ltree, "B", rtree);
        ltree = new BinaryTree(new BinaryTree(), "F", new BinaryTree());
        current = new BinaryTree(ltree, "C", new BinaryTree());
        ltree = btree;
        rtree = current;
        btree = new BinaryTree(ltree, "A", rtree);
        System.out.println(btree.rootData());
        System.out.println(btree.leftSubTree().rootData());
        System.out.println(btree.rightSubTree().rootData());
        System.out.println(btree.leftSubTree().leftSubTree().rootData());
```

```
        System.out.println(btree.rightSubTree().leftSubTree().rootData());
        btree.inorder();
        System.out.println();
        btree.preorder();
        System.out.println();
        btree.postorder();
        System.out.println();
        btree.levelorder();
        System.out.println();
        current = btree.copy();
        current.inorder();
        System.out.println();
        if (btree.equals(current))
            System.out.println("They are the same trees");
        else
            System.out.println("They are different trees");
        if (btree.equals(ltree))
            System.out.println("They are the same trees");
        else
            System.out.println("They are different trees");
    }
}
```

<실행결과 & main( )에서 만들어지는 트리>

```
A
B
C
D
F

Inorder
DBGHEAFC
preorder

ABDEGHCF
preorderIter
ABDEGHCF
DHGEBFCA
ABCDEFGH

Inorder
DBGHEAFC
They are the same trees
They are different trees
```

